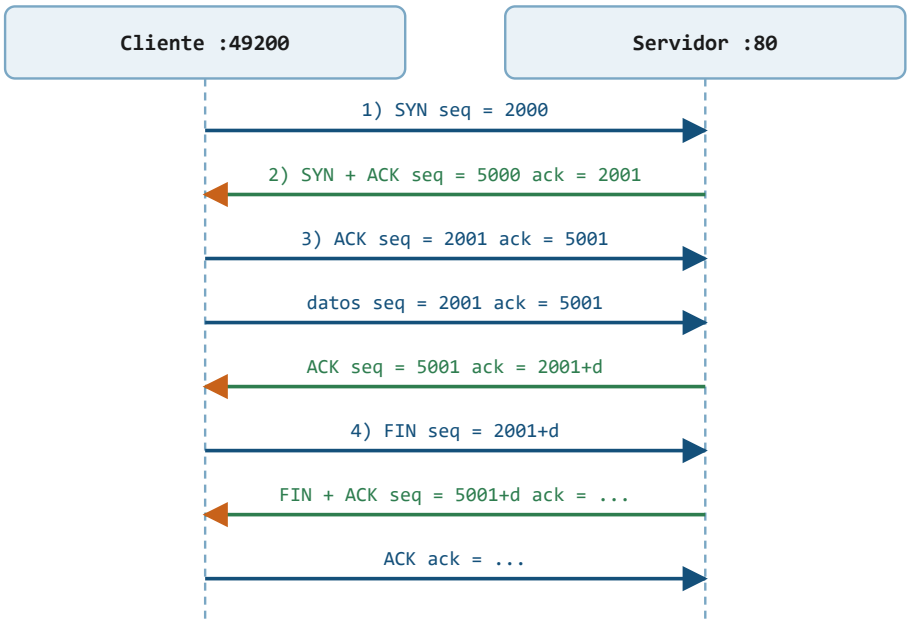
Clase 18 — TCP: establecer y cerrar la conexión

Bloque: C (Transporte) · **Semana:** 6 · **Duración estimada:** 110 min **Correlativa:** Clase 17 (TCP: la cabecera campo por campo) · **Sigue con:** Clase 19 (TCP: cómo se garantiza la fiabilidad) **Material de referencia:** Tanenbaum cap. 6 (§6.3, §6.5) · RFC 9293, RFC 4987, RFC 1122, RFC 7414

🎯 Al terminar esta clase vas a poder

1. Dibujar el **three-way handshake** con números de secuencia concretos y explicar qué se acuerda en cada mensaje.
2. Explicar **por qué 3 vías y no 2**, y por qué el servidor no debe quedar con un socket abierto para siempre.
3. Describir el **SYN flood** y cómo lo mitigan las **SYN cookies** (`RFC 4987`).
4. Dibujar el **cierre de 4 vías** con los números avanzando, y explicar el **semi-cierre** y el **RST**.
5. Recorrer los **11 estados** de la máquina de estados y justificar el **TIME-WAIT de 2×MSL**.
6. Calcular el **RTT** de establecer una conexión y sus variantes (TLS 1.3, 0-RTT, TCP Fast Open).

Three-way handshake y cierre de 4 vias (Examen 2006-12 P4)



> **Prerequisito del examen:** `2006-12 P4` (10 pts — three-way handshake, cierre y TPDUs), `2009-07 P2a` (8 pts — los pasos del handshake).

PARTE 1 — TEORÍA

1.1 Por qué hace falta un handshake

Para mandar datos de forma confiable, los dos extremos tienen que ponerse de acuerdo en **cinco cosas**. Ninguna se puede inventar solo uno de los dos lados:

Variable	Quién la propone	Cómo viaja
ISN del cliente (*Initial Sequence Number*)	el cliente	en el `seq` del SYN
ISN del servidor	el servidor	en el `seq` del SYN+ACK
MSS de cada lado	cada uno	opción MSS en el SYN
SACK permitted, **Window Scale**, **Timestamps**	cada uno	opciones en el SYN
ECN	cada uno	bits CWR/ECE en el SYN

Ojo con la primera fila: los dos ISN no se "acuerdan", se intercambian. Cada extremo **elige el suyo por su cuenta** y el handshake sirve para que cada uno **se entere del del otro**. La `RFC 9293` da la fórmula:

```
ISN = M + F(localIP, localPort, remoteIP, remotePort, clave_secreta)

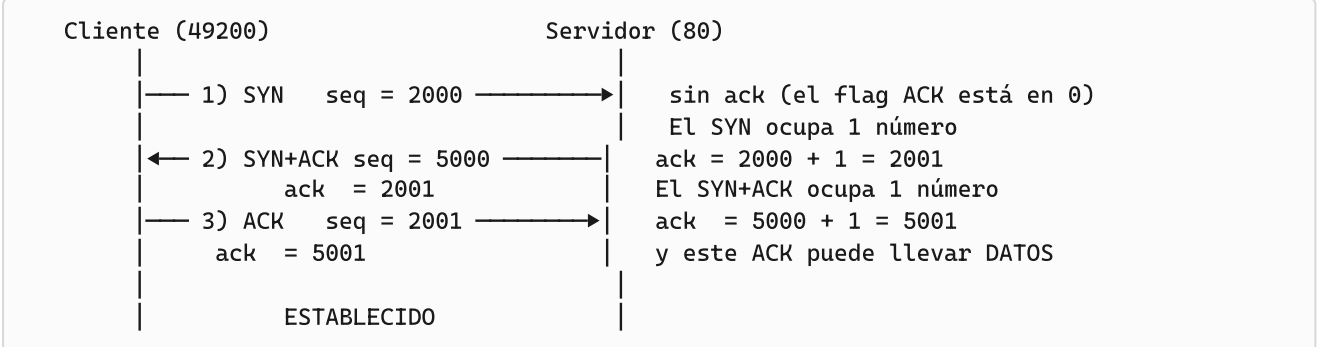
M = un contador que sube al menos cada ~4 microsegundos
F() = función pseudoaleatoria con clave secreta, NO computable de afuera
```

El reloj `M` garantiza que los ISN **no se repitan** dentro de un MSL, y `F()` con clave secreta evita que un atacante **adivine** el número de secuencia (sin eso se puede hacer *spoofing* de segmentos). El ciclo de 32 bits a 4 µs dura **4,55 horas**.

1.2 El three-way handshake, con los números

Voy a usar estos valores en toda la clase, porque después los ejercicios los retoman:

```
Cliente 203.0.113.9, puerto 49200, ISN = 2000 (0x07D0)
Servidor 198.51.100.7, puerto 80,     ISN = 5000 (0x1388)
Ventana anunciada por el cliente: 64240 ; por el servidor: 65535
Opciones en ambos SYN: MSS 1460, SACK permitted, Timestamps, Window Scale
```



Recorrido segmento por segmento, con la cuenta:

#	Seg.	Flags	Long. datos	seq	ack	Columna
1	C→S	SYN	0	**2000**	(0, sin uso)	Empieza la conexión en 2000
2	S→C	SYN+ACK	0	**5000**	**2001**	Confirma que recibió el byte 2000 del cliente
3	C→S	ACK	0 (+datos)	**2001**	**5001**	Confirma que recibió el byte 5000 del servidor

Las dos reglas que hacen que los números cierren: (1) **el SYN consume un número** aunque no lleve datos, por eso el `ack` del segundo segmento es `2000 + 1 = 2001` y no 2000 (el **FIN** también lo hace); (2) el **`ack` es siempre "el próximo byte que espero"**, así que el `seq` del cliente pasa a 2001 y el del servidor a 5001.

> `RFC 9293`, textual: "*the SYN is considered to occur before the first actual data octet of the segment in which it occurs, while the FIN is considered to occur after the last actual data octet*".

El tercer segmento puede llevar datos. Por eso establecer una conexión TCP cuesta **1 RTT y no 2**: el cliente manda su primer request en el mismo ACK que cierra el handshake, con `seq = 2001`.

1.3 ¿Por qué 3 vías y no 2?

Un handshake de 2 vías sería: el cliente manda SYN, el servidor responde SYN+ACK, y se da por establecida. El problema es que **el servidor no tiene ninguna prueba de que el cliente exista**:

```
Con 3 vías                          Con 2 vías
──────────                          ──────────
C ──SYN x──────────► S              C ──SYN x──────────► S
C ◄──SYN+ACK y, x+1─ S              C ◄──SYN+ACK y, x+1─ S
C ──ACK x+1, y+1──► S               (nada más)
↑ el servidor ya sabe que             ↑ el servidor se queda esperando
  hay alguien del otro lado             para siempre

Si se pierde el 3er mensaje:
  el servidor reintenta el SYN+ACK
  → el cliente lo re-responde → se resuelve
```

Los dos objetivos reales del tercer mensaje: (1) **que el servidor tenga evidencia de que hay un cliente real**: con 2 vías queda con un socket esperando un ACK que nunca llega, ocupando memoria (el **TCB**: buffers, números, temporizadores), y un atacante puede abrir miles así solo mandando SYNs; (2) **que los dos extremos synchronizeden sus números antes de mandar datos**, para que ninguna respuesta se pierda.

La apertura simultánea. Existe un caso raro en que **los dos mandan SYN** al mismo tiempo (ambos quieren abrir). Los dos pasan a SYN-SENT, cada uno responde al SYN del otro con un SYN+ACK, y cada uno envía el ACK final. Son **4 mensajes** y termina igual. Es lo que menciona la solución oficial del examen 2006-12 con "intercambio de solicitudes de conexión".

1.4 SYN flood y SYN cookies

El ataque. El atacante manda **ráfagas de SYNs** con **IPs de origen falsificadas**. El servidor, que no puede distinguir un SYN real de uno falso, **crea un TCB por cada uno** (la `RFC 793` exige reservar estado al recibir el SYN), y como las respuestas van a IPs inventadas **nadie contesta**. El backlog se llena y **las conexiones legítimas ya no pueden entrar**. Lo peor: con ~1300 bytes de memoria por TCB, **no hace falta mucho tráfico** para tumbar un servidor.

```
atacante ─── miles de SYNs spoofeados ──→ servidor: backlog lleno  ✗
cliente legítimo ─── SYN ──────────────→        descartado

¿Qué falta? que el servidor pueda DIFERENCIAR un SYN real de uno falso. Con IPs
falsificadas la respuesta no vuelve nunca → no hay prueba de nada.
```

La defensa: las SYN cookies (`RFC 4987` — *TCP SYN Flooding Attacks and Common Mitigations*, **Informational**, agosto 2007, verificado en rfc-editor.org). La idea:

```
SIN cookies                          CON cookies
──────────                           ──────────
SYN → crear TCB (~1 KB)              SYN → NO crear nada
     ↓                                    ↓
SYN+ACK a la IP (puede ser falsa)    calcular una "cookie" con el estado
← queda un TCB por cada ataque        ISN = f(5-tupla, contador, MSS, secreto)
                                     SYN+ACK con ese ISN   ← no hay estado guardado
                                          ↓
                                     llega el ACK → RECALCULAR la cookie
                                          ↓
                                     si coincide → recién AHORA se crea el TCB
```

La chave genial: no hay nada que guarden. El estado que el servidor necesitaría (la 5-tupla, el MSS del cliente, los timestamps) **se codifica dentro del propio número de secuencia** del SYN+ACK. Cuando llega el ACK, el servidor **vuelve a calcular la cookie** desde cero y compara: si coincide, sabe que el cliente recibió el SYN+ACK (o sea, que existe) y recién ahí reserva memoria.

La implementación de referencia trunca el ISN en **5 bits** altos del contador de tiempo (cambia cada 64 s, para que las cookies no se repitan), **3 bits** con uno de 8 MSS comunes y **24 bits** de un *hash* MD5 de las direcciones, los puertos y un secreto del servidor.

Los costos (y por eso no se usa todo el tiempo):

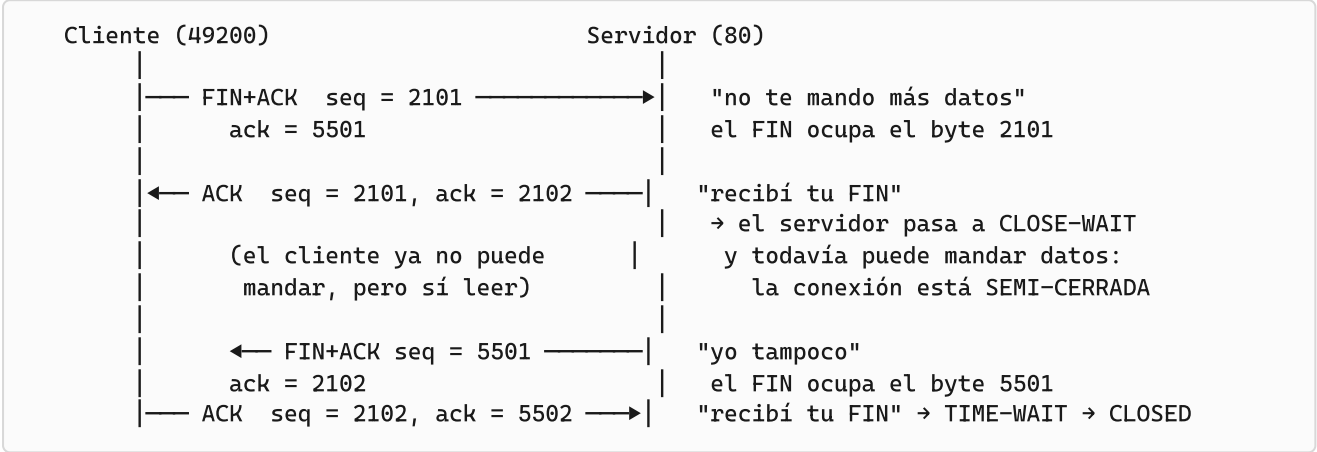
Los costos: se **pierde el Window Scale** (la ventana efectiva cae ÷ 128), **SACK no se puede usar** hasta que el servidor recibe un bloque SACK (lo que limita mucho en flujos unidireccionales), y no se pueden mandar **datos piggyback en el SYN**. Por eso en Linux y Windows están **deshabilitadas por defecto** y se activan **solo bajo ataque**.

> **SCTP resolvió el problema mejor**: handshake de **4 vías** con *cookies* stateless, así que el extremo pasivo tiene **evidencia** de que el iniciador existe antes de reservar nada. La `RFC 4987` lo señala como la lección aprendida.

1.5 El cierre de 4 vías, con los números

TCP es full-duplex, así que cerrar una conexión son **dos cierres independientes**: el de la dirección cliente→servidor y el de la dirección servidor→cliente. **Cada uno necesita 2 mensajes**: un FIN y su ACK.

Retomando los números de §1.2: el cliente mandó 100 bytes de datos (`2001..2100`) y el servidor 500 bytes (`5001..5500`).



La cuenta, que es lo que se corrige:

#	Seg.	Flags	seq	ack	Por qué
1	C→S	FIN+ACK	**2101**	5501	el cliente confirmó los 500 bytes del servidor
2	S→C	ACK	2101	**2102**	el FIN del cliente consumió el 2101
3	S→C	FIN+ACK	**5501**	2102	el servidor confirmó los 100 del cliente
4	C→S	ACK	2102	**5502**	el FIN del servidor consumió el 5501

Por qué 4 y no 3. Cuando el servidor recibe el FIN del cliente **todavía puede tener datos en vuelo** hacia el cliente, así que primero acusa (mensaje 2) y después cierra (mensaje 3). Pero si su FIN viaja **piggyback en el ACK** —porque ya estaba escribiendo— solo hay **3 mensajes**: `FIN`, `FIN+ACK`, `ACK`. Es la variante que muestra la solución oficial del 2006-12.

El semi-cierre (*half-close*): entre el FIN de uno y el del otro, el cliente **ya no puede enviar pero sí recibe**. Es el caso de HTTP/1.x: el cliente manda el request, hace `shutdown(SHUT_WR)` y sigue leyendo la respuesta. Por eso **CLOSE-WAIT** puede durar minutos: no es un error, es diseño.

1.6 RST: el cierre abrupto

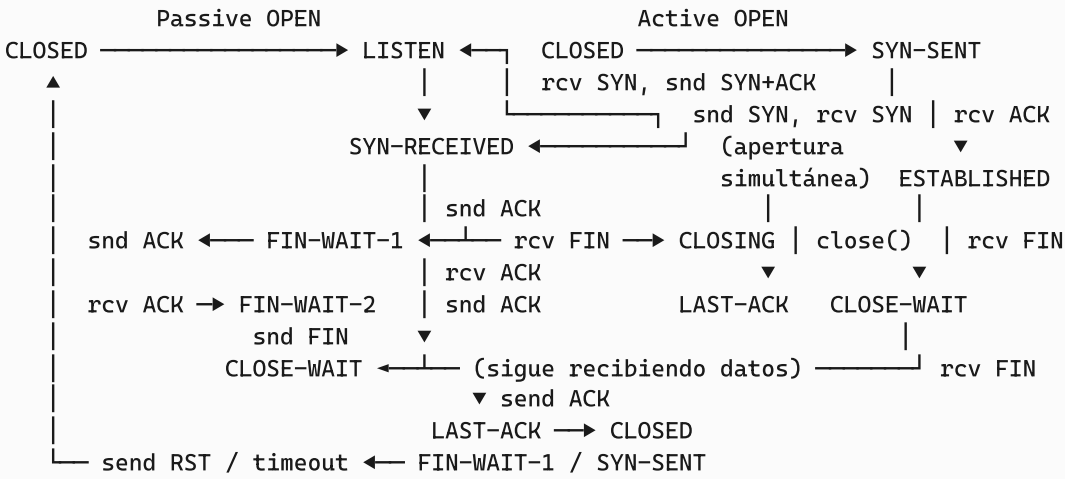
El **RST** (*reset*) es la forma de abortar sin ceremonia. **Cuándo se manda:**

1. **Puerto cerrado**: un SYN a un puerto sin proceso escuchando → el kernel responde `RST, seq 0, ack = seq del SYN`, y el cliente ve **"Connection refused"** (`ECONNREFUSED`).
2. **Datos o FIN sobre una conexión que ya no existe** (o en CLOSED) → `RST` y se descarta todo.
3. **Abortar con datos pendientes**: `close()` con `SO_LINGER` en 0 manda `RST` en vez del cierre ordenado, y **descarta lo no confirmado**.

```
SYN a puerto cerrado:
cliente ──SYN──► servidor :  RST (seq 0, ack = seq del SYN)  → ECONNREFUSED
```

RST no es un cierre limpio: descarta el estado y **pierde los datos no confirmados**. Es lo que pasa cuando un proceso mata un servidor o cuando el SO decide que la conexión lleva demasiado tiempo inactiva (el *keepalive* de la clase 19).

1.7 La máquina de estados: 11 estados



Y el estado **TIME-WAIT**, que falta en el esquema anterior porque se dibuja aparte:

```
LAST-ACK ──send ACK──► TIME-WAIT ──(2 × MSL)──► CLOSED
```

Los 11 estados (los mismos de la `RFC 9293`): `CLOSED`, `LISTEN`, `SYN-SENT`, `SYN-RECEIVED`, `ESTABLISHED`, `FIN-WAIT-1`, `FIN-WAIT-2`, `CLOSE-WAIT`, `CLOSING`, `LAST-ACK`, `TIME-WAIT`.

Los dos estados que se explican menos y siempre se preguntan: CLOSING existe **solo** en el **cierre simultáneo**: los dos mandan FIN antes de recibir el del otro, y cada uno queda ahí hasta que llegue el ACK de su FIN. **TIME-WAIT** es lo que sigue.

1.8 TIME-WAIT: 2 × MSL

MSL es el *Maximum Segment Lifetime*, el tiempo máximo que un segmento puede estar dando vueltas en la red. La `RFC 9293` dice textual: "*For this specification the MSL is taken to be 2 minutes*", y obliga (MUST-13): "*When a connection is closed actively, it MUST linger in the TIME-WAIT state for a time 2xMSL*".

```
MSL = 120 s  →  TIME-WAIT = 2 × 120 = 240 s = 4 minutos
```

El problema que resuelve. Cuando la 4-tupla se reutiliza para una conexión nueva (y se reutiliza constantemente: muchísimos clientes al mismo servidor 80), pueden quedar **segmentos viejos** de la conexión anterior dando vueltas por la red. Si el nuevo SYN del cliente empieza con un número de secuencia que **ya pasó por la red**, el servidor puede confundirlo con datos de la conexión vieja.

```
cliente:   SYN ──────► FIN ──► TIME-WAIT (240 s) ──► CLOSED
red:       ... un duplicado del SYN atrasado 200 s .....................
servidor:                          ◄─── llega tarde: lo aceptaría como dato nuevo ⚠
```

La solución: el lado que cierra primero (TIME-WAIT) **no vuelve a usar esa 4-tupla durante 2 × MSL**. Si durante ese tiempo llega un segmento con números de la conexión vieja, **no coincide** con la conexión nueva y se descarta. Pasados los 2 × MSL ya no puede quedar ningún segmento viejo en la red.

Los dos MSL sirven para cosas distintas: el primero cubre el caso de que el ACK final se haya perdido y el otro extremo reenvíe su FIN (hay que estar ahí para responder); el segundo, los datos rezagados.

Un detalle fino de la `RFC 9293` (MAY-2): en TIME-WAIT se puede **aceptar un SYN nuevo** y reabrir la conexión desde ese estado **si** el ISN asignado es **mayor** que todos los usados en la conexión anterior, y **si** el SYN resulta un duplicado viejo se vuelve a TIME-WAIT. Y con **Timestamps** hay un algoritmo mejor para **reducir** el TIME-WAIT (*Best Current Practice*): los timestamps no se repiten, así que distinguir conexiones viejas sin esperar 4 minutos.

1.9 ⚠️ Actualización a 2026

Sigue igual — de memoria: el **three-way handshake** y sus tres TPDUs (SYN, SYN+ACK, ACK); **el SYN consume un número**; **FIN consume un número**; el **cierre de 4 vías** (y su variante de 3 con FIN piggyback); el **semi-cierre**; el **TIME-WAIT de 2 × MSL**; el **RST**; y **los 11 estados**, que en 2026 son exactamente los mismos.

Cambió o hay que corregirlo:

- `RFC 9293` **(2022, *Internet Standard*) sustituye a la** `RFC 793` y es la referencia de la máquina de estados y del TIME-WAIT. `RFC 1122` sigue definiendo los hosts, y `RFC 7414` (2015) es el **índice de todas las RFCs de TCP**.
- **SYN cookies:** `RFC 4987` (2007, *Informational*). Sigue siendo el mecanismo vigente y lo usan Linux, Windows y BSD. ⚠️ Ojo con el matiz: es un documento **informativo**, o sea **no impone requisitos**, pero es *la* referencia del tema.
- **El MSL es "una decisión de ingeniería"**: la `RFC 9293` toma 2 minutos y dice explícitamente que puede cambiarse con la experiencia. Si te preguntan el número exacto de 4 minutos, es el de RFC 9293.
- **TCP Fast Open** manda datos **dentro del SYN** y ahorra 1 RTT. ⚠️ **El número de RFC no está en `ACTUALIZACIONES.md`: verificá antes de citarlo** (creo que es 7413, pero confirmá).
- **QUIC elimina el three-way handshake de TCP.** El handshake de QUIC es de **1 RTT** (y **0 RTT** con reanudación), va en UDP, y **la conexión se identifica por el Connection ID** (`RFC 9000` §5.1), no por la 4-tupla: eso habilita la **migración de conexión** si cambia la IP o el puerto.
- **Aun así, el modelo TCP de 4-tupla es lo que se enseña y lo que se pregunta.** QUIC no reemplazó a TCP: HTTP/3 es el 21 % del tráfico y HTTP/1.1 sigue siendo el 29 % (Cloudflare Radar, 2025). TCP mueve la enorme mayoría del tráfico de la web, del correo, de SSH y de los juegos.

Consecuencia para el examen: el diagrama del handshake, los números, el cierre de 4 vías y el TIME-WAIT **se preguntan exactamente igual que en 2006**. Si citás `RFC 9293` en lugar de RFC 793, sumás.

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — La secuencia exacta de cada segmento

Enunciado. El cliente (puerto 50000) elige ISN = **1000** y el servidor (puerto 80) ISN = **7000**. La conexión abre, el cliente manda **300 bytes** de datos, el servidor le responde con **1000 bytes**, y después **cada uno cierra su lado**. Escribí **cada segmento** con sus flags, su `seq` y su `ack`.

Resolución, segmento por segmento.

#	Seg.	Flags	Datos	seq	ack	De dónde sale el número
1	C→S	SYN	0	1000	—	ISN del cliente
2	S→C	SYN+ACK	0	7000	**1001**	1000 + 1 (el SYN del cliente)
3	C→S	ACK	0	1001	**7001**	1000 + 1 (el SYN del servidor)
4	C→S	PSH+ACK	**300**	1001	7001	los datos arrancan en 1001
5	S→C	ACK	0	1001	**1301**	1001 + 300 = 1301
6	S→C	PSH+ACK	**1000**	7001	1301	los datos arrancan en 7001
7	C→S	ACK	0	1301	**8001**	7001 + 1000 = 8001
8	C→S	FIN+ACK	0	**1301**	8001	el cliente ya no manda nada más
9	S→C	ACK	0	1301	**1302**	1301 + 1 (el FIN del cliente)
10	S→C	FIN+ACK	0	**8001**	1302	el servidor ya no manda nada más
11	C→S	ACK	0	1302	**8002**	8001 + 1 (el FIN del servidor)
12	C→S	FIN+ACK	0	1302	8002	el cliente cierra **su** lado
13	S→C	ACK	0	1303	8002	1302 + 1
14	S→C	FIN+ACK	0	8002	1303	el servidor cierra **su** lado
15	C→S	ACK	0	1303	**8003**	8002 + 1 → **el cliente entra en TIME-WAIT**

Las tres cuentas que hay que escribir en el examen:

```
Bytes del cliente:  1001 .. 1300   →  próximo esperado: 1301
Bytes del servidor: 7001 .. 8000   →  próximo esperado: 8001
FIN del cliente en 1301 → siguiente 1302 ;  FIN del servidor en 8001 → siguiente 8002
```

Interpretación: el `seq` de cada extremo **solo avanza** cuando ese extremo manda bytes o un SYN/FIN. Un error clásico es poner `ack = 1300` en vez de `1301`: el ACK es **el próximo esperado**.

⚠️ **Errores típicos:** (a) olvidar que **SYN y FIN consumen un número**; (b) poner el `ack` del último byte recibido en vez del próximo; (c) hacer el cierre en **3** segmentos cuando los dos lados mandaron datos en direcciones opuestas (solo van 3 si el segundo FIN va piggyback en el ACK).

2.2 Ejercicio 2 — ¿Qué pasa si se pierde el tercer segmento (el ACK del cliente)?

Enunciado. El SYN del cliente llega, el servidor responde SYN+ACK, y **el ACK del cliente se pierde**. Describí qué estado queda cada uno, qué se retransmite y cuánto cuesta.

Resolución paso a paso.

```
t=0    C ──SYN seq=2000──▶ S        C: SYN-SENT          S: LISTEN
t=RTT  C ◀──SYN+ACK seq=5000, ack=2001── S
                              ✗ (este mensaje se pierde)
       C: ESTABLISHED                  S: SYN-RECEIVED  ← tiene un TCBReservation

t=RTO  S ──SYN+ACK seq=5000, ack=2001──▶ C   (el servidor reintenta)
       C ya está ESTABLISHED y recibe un SYN+ACK duplicado:
       reconoce que es de su propia conexión → responde con el ACK otra vez
t=2RTT C ──ACK seq=2001, ack=5001──▶ S
                              ✓     S: ESTABLISHED
```

Consecuencias, en orden de importancia:

1. **El cliente queda ESTABLISHED y el servidor en SYN-RECEIVED.** El servidor tiene **un TCB reservado que todavía no sabe si es real** (exactamente el agujero que explotan los SYN floods).
2. **El servidor reintenta el SYN+ACK** con su temporizador. Como el cliente ya está establecido, el kernel del cliente ve un **SYN+ACK duplicado de su propia conexión** y **reenvía el ACK** en vez de tratar el SYN como una conexión nueva. **El handshake se completa solo en el segundo intento.**
3. **Costo: 1 RTO extra**, no pérdida de datos. Si el cliente ya mandó datos (lo normal, porque el tercer segmento puede llevarlos), llegan al servidor mientras está en SYN-RECEIVED: **no los descarta**, los reordena en el buffer y los entrega cuando se complete el handshake; si no los ve confirmados, los retransmite.
4. **No es un error del protocolo**, es el comportamiento normal ante pérdida. Por eso el temporizador del SYN+ACK (RTO inicial) es generoso: si se perdiera dos veces, el servidor **cierra la conexión medio abierta** (y con **SYN cookies** ni siquiera hay TCB que cerrar).

⚠️ **Error típico:** decir "el cliente se queda esperando el ACK del servidor". No: el cliente ya está establecido porque **él fue el que mandó el último mensaje del handshake**.

2.3 Ejercicio 3 — ¿Cuánto RTT cuesta establecer una conexión?

Enunciado. Calculá el RTT total desde que el cliente decide abrir la conexión hasta que puede mandar datos de aplicación, en cuatro escenarios: (a) TCP solo; (b) TCP + **TLS 1.3**; (c) TLS 1.3 con **0-RTT** (reanudación); (d) **TCP Fast Open** + TLS 1.3. Suponé RTT de 40 ms.

Escenario	Vuelos de ida	RTT	Total	¿Datos en el 1.er vuelo?
(a) TCP solo	SYN · SYN+ACK+ACK	**1**	40 ms	sí (van en el 3.er segmento)
(b) TCP + TLS 1.3	SYN · SYN+ACK · ClientHello+ACK · ServerHello… · fin	**1**	**40 ms**	no
(c) TLS 1.3 0-RTT	como (b) pero con *early data*	**1**	40 ms	**sí** (datos antes del handshake)
(d) TCP Fast Open + TLS 1.3	SYN+datos · SYN+ACK+datos	**0**	**0 ms**	sí

La aritmética y el razonamiento de cada fila:

- **(a) TCP solo = 1 RTT.** El cliente abre el flujo de un lado y recibe la respuesta: un RTT. Los datos de aplicación viajan **en el tercer segmento** (el ACK), así que no cuesta nada más.
- **(b) TLS 1.3 = 1 RTT total, no 2.** El trick es que el **ClientHello viaja en el tercer segmento del handshake TCP**, que ya estaba en vuelo. O sea: la `RFC 8446` promete "1-RTT" **contado desde el ClientHello**, y como ese ClientHello piggyback del ACK de TCP, **desde el SYN son 1 RTT**. Por eso HTTPS con TLS 1.3 se ve tan rápido.
- **(c) 0-RTT** requiere **reanudar una sesión** (con PSK) y un **token anti-replay** (`RFC 8470`), sin el cual el servidor no puede aceptarlo: si lo acepta a ciegas, un atacante **repite** una petición vieja. El payload va en el primer vuelo; el costo es que la aplicación tiene que aceptar el replay.
- **(d) TCP Fast Open = 0 RTT** para el que ya tiene *cookie*: manda los datos **dentro del SYN**. ⚠️ **El número de RFC no está en `ACTUALIZACIONES.md` : verificá** (creo que es 7413). Sin cookie, el servidor la pide en el SYN+ACK y el cliente reenvía los datos: **1 RTT y medio**.

Conclusión para el examen: el detalle que más se pregunta es que **TLS 1.3 no agrega un RTT** al handshake TCP porque **el ClientHello piggyback del ACK de TCP**, y que **0-RTT existe pero obliga a renunciar a la protección contra repetición**.

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2006-12, Pregunta 4 (10 puntos) — Three-way handshake y cierre

> **Fuente:** Examen `eirc0612` / `sirc0612` — 26 de diciembre de 2006.

Enunciado (texto del examen)

> Describa el proceso por el cual se establece una conexión TCP (three-way handshake), y cómo se cierra la misma. ¿Qué TPDU se intercambian en los mismos?

Solución oficial (verbatim, resumida)

> El establecimiento se realiza con un three-way handshake: un SYN; si el puerto está disponible, un SYN-ACK, y finalmente un ACK, y de no estar se envía un RST. También puede realizarse por intercambio de solicitudes de conexión (b), aunque no es común. El número inicial de secuencia se acuerda en el momento de la conexión. No es 0. Desconexión: TCP supone la existencia de dos conexiones simplex que se desconectan independientemente. El envío de un paquete con flag FIN implica que yo no deseo enviar más datos al otro extremo. En general se necesitan 4 segmentos para cerrar una conexión, aunque pueden ser 3: FIN / ACK / FIN / ACK y FIN / FIN+ACK / ACK. Para evitar el problema de los dos ejércitos se colocan timers que indican que si ocurren 2 time_outs sin recibir el ACK del FIN se da por finalizada la conexión.

Análisis de la solución oficial

Lo que está bien: los tres TPDUs del handshake y el **RST** cuando el puerto no está disponible; la idea de **dos conexiones simplex**; el **cierre de 4 o 3 segmentos**; y la apertura simultánea.

Tres cosas que hay que corregir o precisar (y que suman puntos si las señalás):

1. **"El número inicial de secuencia se acuerda en el momento de la conexión."** Impreciso: los dos ISN **no se acuerdan, se intercambian**: cada extremo elige el suyo (aleatorio y con clave secreta, para que nadie lo prediga) y el handshake sirve para que cada uno **se entere del del otro**.
2. **"Para evitar el problema de los dos ejércitos se colocan timers … si ocurren 2 time_outs."** Eso **no resuelve** el problema de los dos ejércitos: **abandonar la conexión al segundo timeout resuelve los ENEMIGOS, no los aliados**. La solución canónica es el **TIME-WAIT de 2 × MSL**, que impide que los dos ejércitos se crucen.
3. **"El del otro extremo notará que no hay nadie escuchando y cortará."** Eso es lo que pasa con un **RST** (el puerto se cerró). En el cierre ordenado con FIN, el otro extremo **sigue escuchando** hasta

mandar su propio FIN.

Mi resolución, completa

Establecimiento: el cliente manda **SYN** con su ISN en `seq` (sin `ack`); el servidor, si el puerto está libre, responde **SYN+ACK** con su ISN en `seq` y `ack = ISN del cliente + 1`; el cliente manda **ACK** con `seq = su ISN + 1` y `ack = ISN del servidor + 1`, y **ese tercer segmento puede llevar datos**. Si el puerto no está libre, el servidor responde **RST**.

Cierre (4 vías): TCP es **full-duplex**, así que hay **dos conexiones simplex** que se cierran por separado: el cliente manda **FIN+ACK**, el servidor responde **ACK** (acá el cliente ya está en **semi-cierre**), el servidor manda su **FIN+ACK** y el cliente cierra con un **ACK** final, quedando en **TIME-WAIT 2 × MSL**. Si el servidor tenía datos pendientes, su FIN viaja **piggyback** en el ACK y son **3 segmentos**.

TPDUs intercambiados: `SYN`, `SYN+ACK`, `ACK`, `FIN`, `ACK`, `FIN`, `ACK` (y `RST` en cualquier punto para abortar).

> **Tope para 10 puntos:** los tres TPDUs del handshake + el RST, el cierre de 4 vías con el **semi-cierre**, el **TIME-WAIT de 2 × MSL** justificado con el problema de los dos ejércitos, y los TPDUs del cierre.

3.2 Examen 2009-07, Pregunta 2a (8 puntos) — Los pasos del handshake

> **Fuente:** Examen `eirc0907` / `sirc0907` — 29 de julio de 2009.

Enunciado (texto del examen)

> a) Describa los pasos del "Three Way Handshake" para el inicio de una conexión TCP.

Solución oficial (verbatim)

> Paso 1: el host cliente envía un segmento SYN al servidor - especifica el nro. de sec. inicial - sin datos. Paso 2: el host servidor recibe el SYN, responde con un segmento SYN-ACK sin datos. El servidor reserva buffers, especifica el nro. de sec. inicial del servidor. Paso 3: el cliente recibe el SYN-ACK, responde con un segmento ACK, que puede tener datos.

Mi resolución, ampliada

La solución oficial es correcta y completa. Lo que hay que agregarle para los 8 puntos es **por qué tres y no dos** y **qué se negocia**:

Paso 1 — cliente → servidor: SYN. El cliente elige su ISN, lo pone en `seq`, manda `ack = 0` (con ACK en 0 el campo Acknowledgment **se ignora**) y **no lleva datos**. Aprovecha para anunciar en las **opciones** su

MSS, SACK, Window Scale y Timestamps.

Paso 2 — servidor → cliente: SYN+ACK. El servidor **reserva TCB y buffers**, elige **su propio ISN** y responde con `seq = ISN servidor` y `ack = ISN cliente + 1`, más **sus opciones**. **Aquí todavía no hay conexión**: está en SYN-RECEIVED y no sabe si el cliente existe de verdad.

Paso 3 — cliente → servidor: ACK. `seq = ISN cliente + 1`, `ack = ISN servidor + 1`, y **puede llevar datos** (es el request HTTP). Con esto los dos saben los números del otro y **la conexión queda establecida**: el cliente pasa a ESTABLISHED y el servidor también.

Y el "cuarto paso" que no está en la solución oficial: si el servidor no tiene ese puerto abierto, responde **RST** y el `connect()` del cliente falla con `ECONNREFUSED` al instante. Esa es la razón por la que el SYN flood duele tanto: **el SYN reserva estado antes de tener ninguna prueba**.

PARTE 4 — CIERRE

📌 Resumen en 10 líneas

1. El handshake sirve para que **cada extremo se entere del ISN y de las opciones del otro**: los ISN **se intercambian, no se acuerdan**, y son aleatorios con clave secreta.
2. **SYN y FIN consumen un número de secuencia**; por eso los ACKs son `ISN + 1`.
3. El **tercer segmento** (ACK) **puede llevar datos**, y por eso establecer una conexión TCP cuesta **1 RTT**.
4. **3 vías y no 2** porque el servidor necesita **evidencia de que el cliente existe** antes de reservar un TCB.
5. **SYN flood** = SYNs con IP falsificada que llenan el backlog; las **SYN cookies** (`RFC 4987`) lo evitan **codificando el estado en el ISN** y creándolo recién cuando el ACK valida la cookie.
6. El **cierre de 4 vías** cierra **dos conexiones simplex** por separado: FIN, ACK, FIN, ACK; y son **3 segmentos** si el segundo FIN va piggyback en el ACK.
7. Entre el FIN de uno y el del otro hay **semi-cierre**: se puede seguir leyendo.
8. El **RST** es el cierre abrupto: puerto cerrado, datos sobre conexión inexistente, o `close()` con linger 0. Descarta lo no confirmado.
9. La máquina de estados tiene **11 estados**, y **TIME-WAIT = 2 × MSL = 240 s** (MSL = 2 min en `RFC 9293`) impide que un segmento viejo se confunda con uno de la conexión nueva.
10. **QUIC elimina el handshake de 3 vías** y usa el **Connection ID**, pero **TCP con 4-tupla sigue siendo lo que se enseña y lo que se pregunta**.

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. En el three-way handshake, con ISN del cliente 2000 e ISN del servidor 5000, ¿cuáles son los `seq` y `ack` de los tres segmentos?

▶ Respuesta

2. ¿Por qué el handshake tiene 3 mensajes y no 2?

▶ Respuesta

3. ¿Qué es una SYN cookie y por qué funciona?

▶ Respuesta

4. En el cierre de 4 vías, ¿cuántos números consume cada FIN y por qué?

▶ Respuesta

5. ¿Por qué existe el estado TIME-WAIT de 2 × MSL?

▶ Respuesta

6. ¿Cuánto cuesta establecer una conexión TCP con TLS 1.3, y por qué no son 2 RTT?

▶ Respuesta

7. ¿Qué es el semi-cierre y en qué caso típico aparece?

▶ Respuesta

8. ¿Cuándo se manda un RST y qué efecto tiene?

▶ Respuesta

🔗 Referencias

- **Examen** `eirc0612` **/** `sirc0612` — P4 (10 pts, three-way handshake, cierre y TPDUs)
- **Examen** `eirc0907` **/** `sirc0907` — P2a (8 pts, los pasos del handshake)
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 6 — §6.3 (*Segmentos TCP*)
- `RFC 9293` — *TCP* (2022, *Internet Standard*): máquina de estados, TIME-WAIT (MUST-13, MAY-2), MSL, ISN
- `RFC 4987` — *TCP SYN Flooding Attacks and Common Mitigations* (2007, *Informational*): SYN flood, SYN cache, SYN cookies, SCTP
- `RFC 1122` — requisitos de los hosts · `RFC 7414` — índice de las RFCs de TCP
- **TCP Fast Open** — datos en el SYN: ⚠️ **el número de RFC no figura en** `ACTUALIZACIONES.md`**: verificá**
- `RFC 8446` → `RFC 9846` — TLS 1.3 (1-RTT y 0-RTT) · `RFC 8470` — tokens anti-replay
- `RFC 9000` **/** `RFC 9001` **/** `RFC 9114` — QUIC, TLS 1.3 sobre QUIC y HTTP/3: handshake de 1 RTT, Connection ID y migración
- **Clase siguiente:** 19 — TCP: cómo se garantiza la fiabilidad